

FUNDAMENTALS OF PROGRAMMING

Lecture 14: POINTERS

Instructor: Dr Aqib Perwaiz

Pointers



Pointers

- It provides a way of accessing a variable without referring to its name.
- A pointer is just the memory address of a variable
- A *pointer variable* is just a variable in which we can store different memory addresses
- The mechanism used for this is the **address** of the variable.

Address of variable

- Each variable occupies some bytes in memory (according to its size)
- Each byte of memory has a unique address so that it can be accessed (just like the address of our homes)
- Variable names are actually titles given to these addresses
 - ▣ Such as the 'white house'
- When we use a variable, the compiler fetches the value placed at its address or writes a new value to that address

Pointers

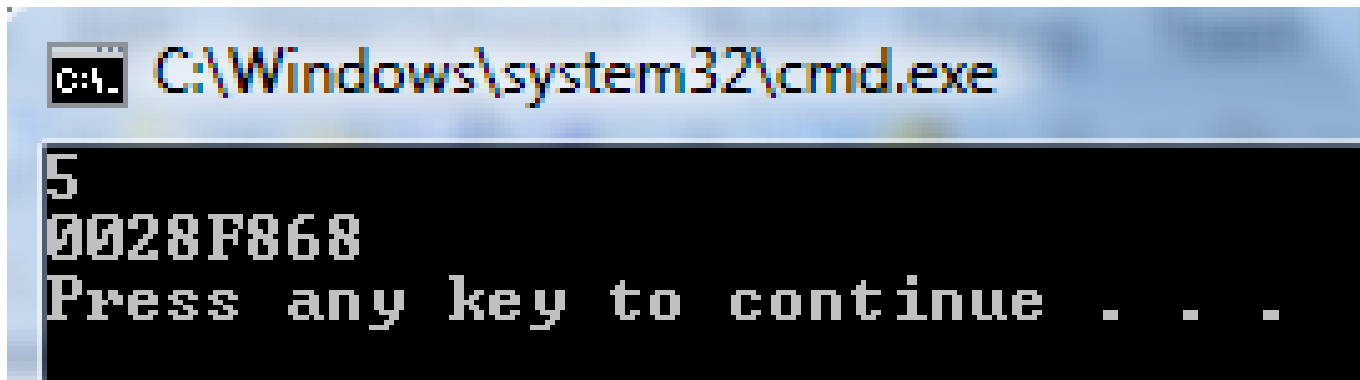
- Pointers are just variables storing numbers –but those numbers are memory addresses
- A pointer that stores the address of some variable *x* is said to ***point to x***.
- A pointer is declared by putting a star (or '*') before the variable name.
- To access the **value** of *x*, we'd need to *dereference* the pointer.

- Roughly speaking, "*" means "the variable located at the address",
- "&" means "the address of the variable"

Address of variable

- How can we find the address of that variable?
- The “&” (Ampersand) operator returns the address of a variable

```
cout << a << endl;  
cout << &a << endl;
```



A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\system32\cmd.exe'. The command prompt displays the output of a program: the number '5' on the first line, the memory address '0028F868' on the second line, and the text 'Press any key to continue . . .' on the third line.

```
C:\Windows\system32\cmd.exe  
5  
0028F868  
Press any key to continue . . .
```

Motivation for using Pointers

- ❑ To return more than one value from a function.
- ❑ To pass arrays and strings more conveniently from one function to another.
- ❑ To manipulate arrays more easily by moving pointers to them.

Motivation for using Pointers

- In more advanced programming, such as
 - ▣ To create complex data structures, such as linked lists and binary trees
 - ▣ For dynamic memory allocation and de-allocation

Pointers

- Declaration of pointer to an int

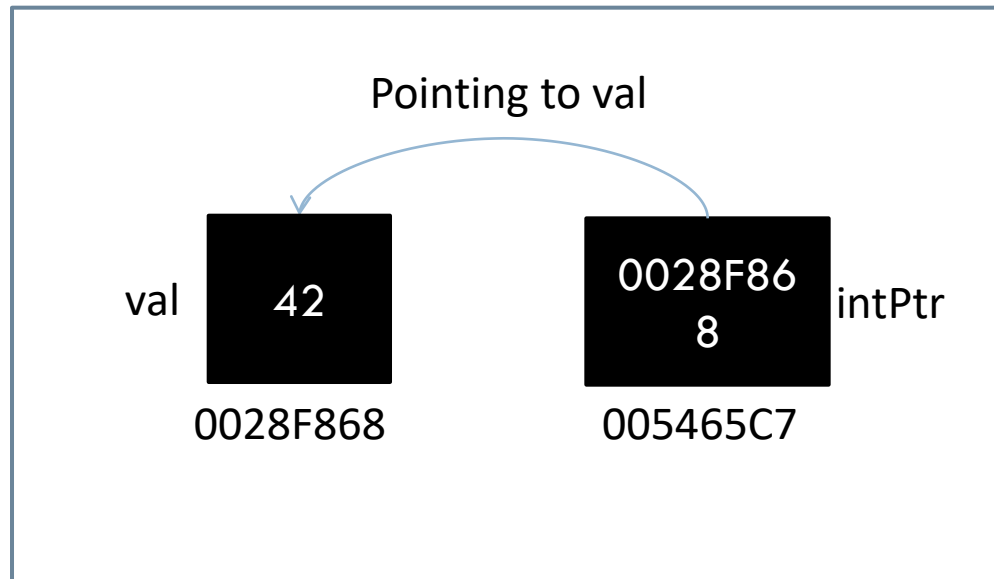
```
int* intPtr;
```

- Initializing a pointer

```
int val = 42;  
int* intPtr = &val;
```

- The ampersand (or '&') before val means "take the address of val".
- The pointer intPtr is assigned the address of val, which is another way of saying that intPtr now points to val.

Pointers



Pointers

- If we print out the value of `intPtr`, we'll get the address of `val`.
- To get the data that `intPtr` actually points at, we have to *dereference* `intPtr` with the *unary star operator* `*`

```
int val = 42;  
int *intPtr = &val;  
cout << "&val: " << &val << endl; // address of val  
cout << "intPtr: " << intPtr << endl; // ..again, address  
cout << "*intPtr: " << *intPtr << endl; // displays 42
```

- Note that the star (or `*`) operator has many uses. When used in a binary context, it can mean multiplication. When used in a variable declaration, it means "this variable is a pointer". When used in an expression as a single-argument operator, it can mean "dereference". With so many uses, it's easy to get confused.

Pointers

- Since intPtr points to val, any changes that we make to val will also show up when we dereference intPtr:

```
int val = 42;
int* intPtr = &val;
cout << "val: " << val << endl; // displays 42
cout<<"*intPtr: "<<*intPtr<<endl;//displays 42
val = 999;
cout << "val: " << val << endl;// displays 999
cout << "*intPtr: " << *intPtr << endl; //
displays 999
```

Pointers

- You can declare a pointer to any type, not just int:

```
char ch= 'H';
```

```
char* chPtr = &ch;
```

```
cout << "ch: " << ch<< endl;
```

```
cout << "chPtr: " << chPtr << endl;
```

```
cout << "*chPtr: " << *chPtr << endl;
```

```
*chPtr = 'I';
```

```
cout << "*chPtr: " << *chPtr << endl;
```

```
cout << "ch: " << ch<< endl;
```

Pointers

```
float val = 1.43;
```

```
float* fltPtr = &val;
```

```
cout << "val: " << val << endl;      // 1.43
```

```
cout << "val: " << fltPtr << endl;    // address
```

```
cout << "*fltPtr: " << *fltPtr << endl; // 1.43
```

```
*fltPtr = 3.1416;
```

```
cout << "*fltPtr: " << *fltPtr << endl; //3.1416
```

```
cout << "val: " << val << endl;      // 3.1416
```

Example

```
int main ( )
{ int x=4, y=7;
  int *px, *py;
  px = &x; //int *px=&x;
  py = &y; //int *py=&y;
  cout << "x = " << x<<"y ="<<y;
  *px = *px + 10;
  *py = *py + 10;
  cout << "x = " << x<<"y ="<<y;
  return 0;
}
```



```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int * ptr_a, * ptr_b;
```

```
    int num_c = 4, num_d = 7;    //group A
```

```
    int num_c = 7, num_d = 4;    //group B
```

```
    ptr_a = &num_c;
```

```
    ptr_b = ptr_a;
```

```
    cout << *ptr_a << " " << *ptr_b << "\n";
```

```
    ptr_b = &num_d;
```

```
    cout << *ptr_a << " " << *ptr_b << "\n";
```

```
    *ptr_a = *ptr_b;
```

```
    cout << *ptr_a << " " << *ptr_b << "\n";
```

Output:

4 4

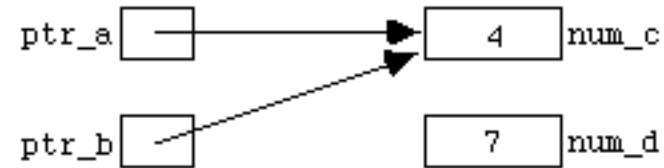
4 7

7 7

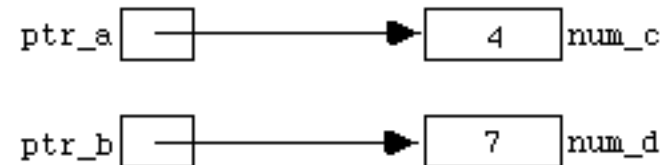
7 7

Diagrammatically

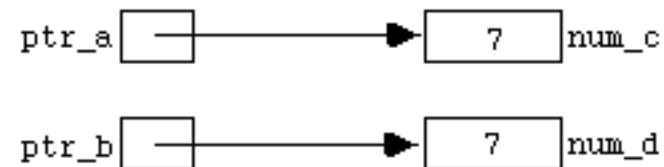
```
ptr_a = &num_c;  
ptr_b = ptr_a;
```



```
ptr_b = &num_d;  
cout << *ptr_a << " " << *ptr_b << "\n";
```



```
*ptr_a = *ptr_b;  
cout << *ptr_a << " " << *ptr_b << "\n";  
cout << num_c << " " << *&*&num_c;
```



Note that "*" and "&" are in a certain sense complementary operations; "&*&*num_c" is simply "num_c".

Exercises

```
int main (void)
{
    int a;
    int *ptr1;
    char c;
    char *ptr2;
    int fred;
    ptr2=&c;           //ok. Char pointer init with add of a char
    ptr1=&fred;        //ok. int pointer init with the address of
                        //an int
    ptr1=&a ;          //ok. int pointer init with the address of
                        //an int
}
```

Exercises

```
ptr2=&fred; // NO. cannot assign the address of an int to a char
ptr2=&'#'; // NO. cannot take the address of an implicit constant
ptr2=&25; // NO. cannot take the address of an implicit constant
ptr2=&(a +3); // NO. cannot take the address of an expression
}
```

Exercises

```
int main()
{ int c,a=10;
  int *p1=&a;
  c=*p1; //equivalent the expression "c=a"
  *p1=*p1**p1; //equivalent the expression "a=a*a"
  (*p1)++; //equivalent the expression "a++"
  c=*&a; //equivalent the expression "c=a"
}
```

Exercises

```
int a = 3;  
int b = 4;  
int *pointerToA = &a;  
int *pointerToB = &b;  
int *p = pointerToA;  
p = pointerToB;  
cout<< a << b << *p; // Prints 344
```

Pointers and Arrays

- In c++, you can treat pointers and arrays very similarly:

```
int array[ ] = {3,1,4,1,5,9};
int* arrayPtr = array;
cout<<"array[0]: " <<array[0] << endl;//displays 3
cout<<"arrayPtr[0]: "<<arrayPtr[0]<<endl;//displays 3
arrayPtr++;
cout<<"arrayPtr[0]: "<<arrayPtr[0]<<endl; // displays 1
array++; // error: arrays are const
```

Pointers and Arrays

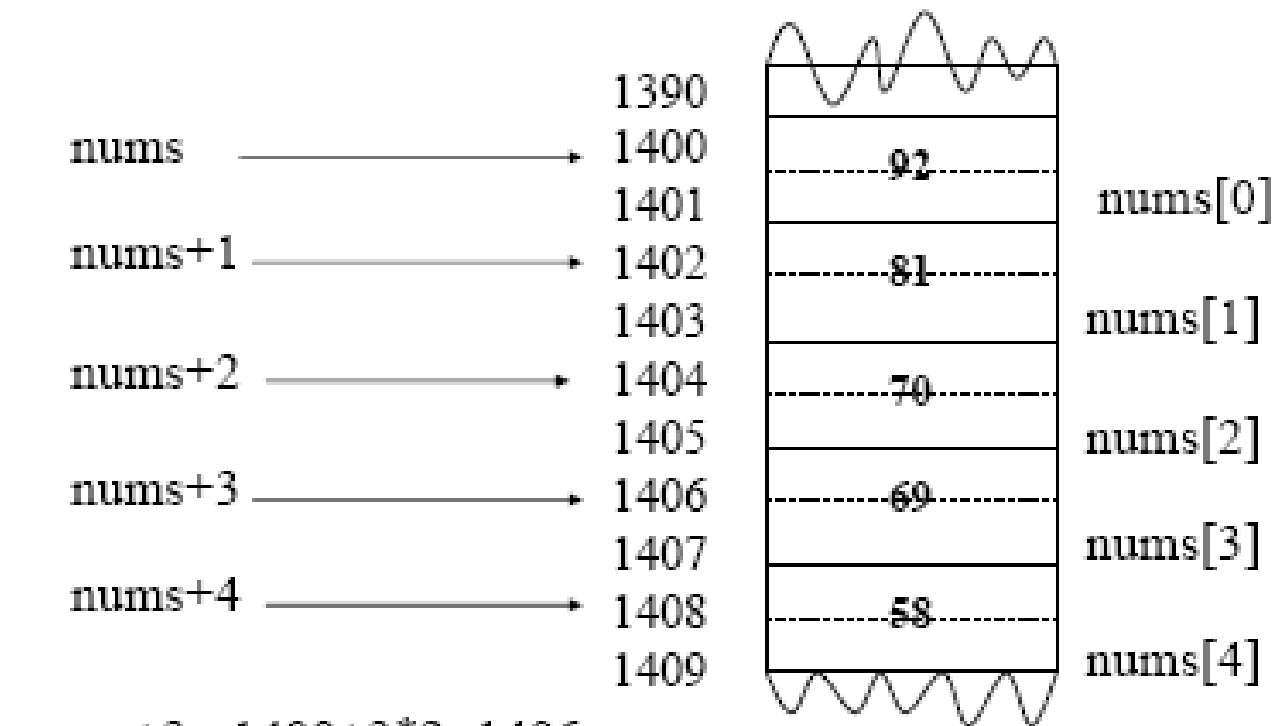
- Arrays and pointers may be used interchangeably, but you can change what pointers point at, whereas arrays will always "point" to the same thing.

Pointers and Arrays

// Using pointers to print out values from array

```
int main ( )  
    {int nums [5] = {92,81,70,69,58};  
      int dex ;  
      for ( dex = 0; dex <5; dex++ )  
          cout << *(nums + dex)<<" ";  
    }
```

□ *** (nums + dex)** is same as **nums [dex]**.



Pointers and Arrays

```
int * ptr; /* pointer to int (or int
array) */
char *chptr; /*pointer to char(or char
array)*/
int table[3]= {5, 6, 7}; /* array */
ptr = table; //&table[0]//*assign array
address to pointer*/
cout << *ptr; /*prints 5*/
cout << *(ptr+1); /* prints 6 */
```

Pointers and Functions

```
/* test function that returns two values */
void rets2 (int* , int* ); /*prototype*/
int main ( )
    {int x, y; /* variables*/
      rets2 ( &x, &y ); /*get values from
function*/
      cout <<"First is = "<< x<<"Second is = "<<y;
    }
/* returns two numbers */
void rets2 ( int* px, int* py )
    { *px= 3; /* set contents of px to 3 */
      *py = 5; /* set contents of px to 5 */
    }
```

Pointers and Referencing

- Reference and pointers have a more or less similar function. However, there are some fundamental differences in how they work.
- Reference is an alias to the variable, while pointer gives the address of the variable.
- References basically function like pointers that are dereferenced every time they are used.

Pointers and Referencing

- However, because you cannot directly access the memory location stored in a reference (as it is dereferenced every time you use the reference), you cannot change the location to which a reference points, whereas you can change the location to which a pointer points.

Acknowledgements

1. Deitel and Deitel: C++ How to Program, 7th Edition, Prentice Hall Publications
2. Robert Lafore: Object-Oriented Programming in C++, Fourth Edition, December 2001, Sams Publishing .
3. A Structured Programming Approach Using C++ by Behrouz A. Forouzan
4. www.cplusplus.com